Gaussian Processes II

Gaussian Process Classification

Edward Milsom

Bayesian Machine Learning • CM32024 • 2025–26 • Lecture 22

Department of
Computer Science

UNIVERSITY OF
BATH

In this Episode ...

- Gaussian Process recap
- GP classification leads to **intractable posteriors**
- **Variational Inference** recap
- VI for GP classification
- All the nitty-gritty details

Gaussian Process Regression Recap

Gaussian Prior over Functions

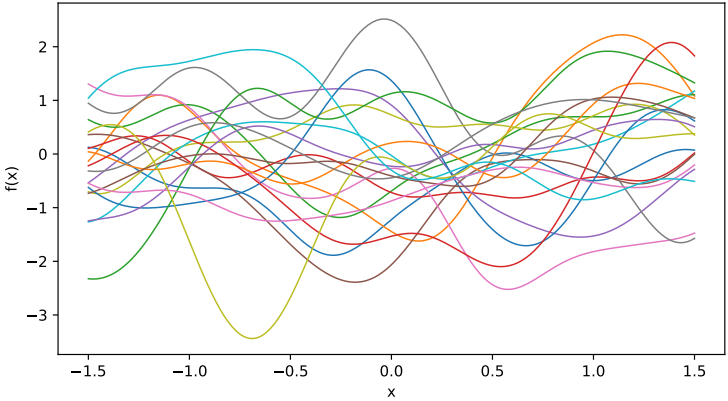
- Let $\mathbf{f} = [f(\mathbf{x}_1), \dots, f(\mathbf{x}_N)]^T$ be the (unknown) function values at inputs $\{\mathbf{x}_n\}_{n=1}^N$
- A **Gaussian Process prior over functions** assumes that $\mathbf{f}$ is distributed according to a **multivariate Gaussian**:

$$\begin{pmatrix} f(\mathbf{x}_1) \\ \vdots \\ f(\mathbf{x}_N) \end{pmatrix} \sim N\left(\begin{pmatrix} m(\mathbf{x}_1) \\ \vdots \\ m(\mathbf{x}_P) \end{pmatrix}, \begin{pmatrix} k(\mathbf{x}_1, \mathbf{x}_1) & \cdots & k(\mathbf{x}_1, \mathbf{x}_N) \\ \vdots & \ddots & \vdots \\ k(\mathbf{x}_N, \mathbf{x}_1) & \cdots & k(\mathbf{x}_N, \mathbf{x}_N) \end{pmatrix} \right)$$

- $m(\cdot)$ is the **mean function** (often set to zero for simplicity).
- $k(\cdot, \cdot)$ is the **covariance function**, or **kernel**, which specifies **how correlated function values are** at different inputs.

GP Prior Samples - Squared Exponential Kernel

- Each curve is a **sample** from the prior $p(\mathbf{f})$ for **equally spaced inputs** $\mathbf{x}$.
- `plt.plot(x, multivariate_normal(zeros(N), K))`



Closed-form posterior (Gaussian noise observations)

- For observed data $\mathbf{X}, \mathbf{y}$, our **joint distribution** is

$$\begin{pmatrix} \mathbf{y} \\ \mathbf{f}_* \end{pmatrix} \sim N\left(\begin{pmatrix} \mathbf{m} \\ \mathbf{m}_* \end{pmatrix}, \begin{pmatrix} \mathbf{K}(\mathbf{X}, \mathbf{X}) + \sigma^2_{\text{noise}}\mathbf{I} & \mathbf{K}(\mathbf{X}, \mathbf{X}_*) \\ \mathbf{K}(\mathbf{X}_*, \mathbf{X}) & \mathbf{K}(\mathbf{X}_*, \mathbf{X}_*) \end{pmatrix}\right)$$

- Using the **partitioned Gaussian conditional formula**, we get:

$$\mathbf{f}_* | \mathbf{y} \sim N(\boldsymbol{\mu}_{*|y}, \boldsymbol{\Sigma}_{*|y})$$

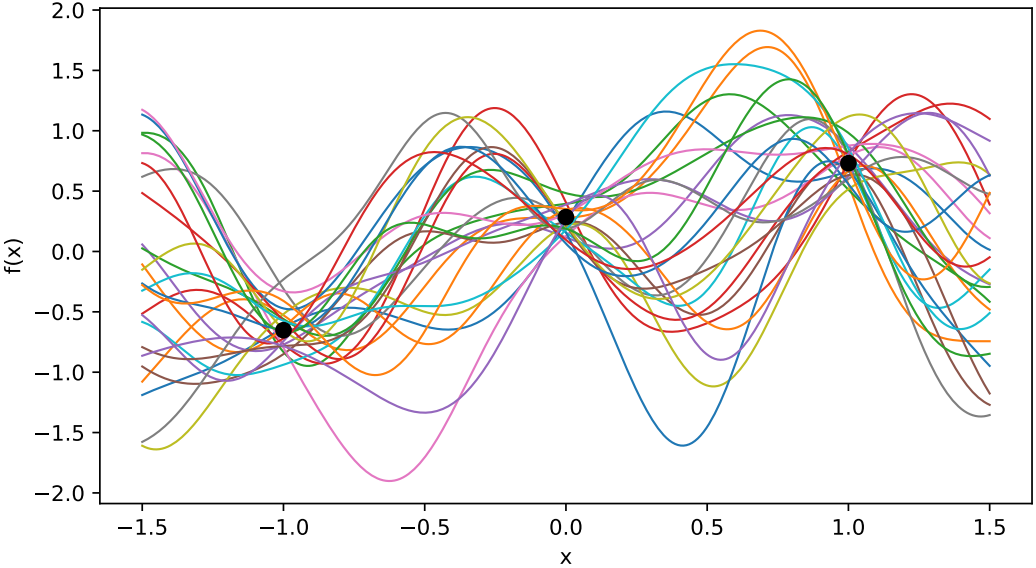
where

$$\boldsymbol{\mu}_{*|y} = \mathbf{m}_* + \mathbf{K}(\mathbf{X}_*, \mathbf{X})[\mathbf{K}(\mathbf{X}, \mathbf{X}) + \sigma^2_{\text{noise}}\mathbf{I}]^{-1}(\mathbf{y} - \mathbf{m})$$

$$\boldsymbol{\Sigma}_{*|y} = \mathbf{K}(\mathbf{X}_*, \mathbf{X}_*) - \mathbf{K}(\mathbf{X}_*, \mathbf{X})[\mathbf{K}(\mathbf{X}, \mathbf{X}) + \sigma^2_{\text{noise}}\mathbf{I}]^{-1}\mathbf{K}(\mathbf{X}, \mathbf{X}_*)$$

- When using noiseless or Gaussian noise observations, and a GP prior, the **posterior is also a GP**!

GP posterior samples (Gaussian noise observations)

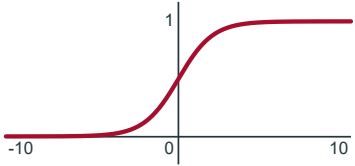


GP Classification — non-conjugate likelihoods

Binary Classification - Bernoulli Likelihood

- Task: Given inputs $\{\mathbf{x}_n\}_{n=1}^N$, predict class labels $y_n \in \{0, 1\}$.
- But GPs learn continuous functions $f(\mathbf{x}) \in \mathbb{R}$, not discrete outputs.
- Solution: Treat f_n as **latent function values**, and squash them through the **logistic function** to give probability of class 1:

$$\sigma(f_n) = \frac{1}{1 + \exp(-f_n)}$$



- Then use a **Bernoulli likelihood**:

$$p(y_n | f_n) = \text{Bernoulli}(y_n | \sigma(f_n)) = \sigma(f_n)^{y_n}(1 - \sigma(f_n))^{1-y_n}$$

Bernoulli Likelihood is Non-Conjugate

- To make predictions, we need the posterior over the latent:

$$p(\mathbf{f}|\mathbf{y}) = \frac{p(\mathbf{y}|\mathbf{f})p(\mathbf{f})}{p(\mathbf{y})}$$

- But with Bernoulli likelihood $p(\mathbf{y}|\mathbf{f})$, the evidence is **intractable**:

$$p(\mathbf{y}) = \int p(\mathbf{y}|\mathbf{f})p(\mathbf{f})d\mathbf{f}$$

- $p(\mathbf{y}|\mathbf{f})$ is not Gaussian, therefore **not conjugate** with the Gaussian prior $p(\mathbf{f})$.
- Must approximate the posterior. Today, we will use **Variational Inference**.

GP Classification using Variational Inference

Variational Inference Recap

- We seek an intractable posterior $p(\mathbf{f}|\mathbf{y})$.
- Introduce an approximate posterior $q(\mathbf{f};\boldsymbol{\theta})$ with parameters $\boldsymbol{\theta}$.
- $q(\mathbf{f};\boldsymbol{\theta})$ should be easy to manipulate and sample from (e.g. Gaussian).
- Minimise the **Kullback-Leibler (KL) divergence** between $q(\mathbf{f};\boldsymbol{\theta})$ and $p(\mathbf{f}|\mathbf{y})$:

$$\boldsymbol{\theta} = \arg\min_{\boldsymbol{\theta}} \mathrm{KL}(q(\mathbf{f};\boldsymbol{\theta})||p(\mathbf{f}|\mathbf{y}))$$

- KL divergence is a kind of distance measure between distributions (always positive, zero if and only if $q = p$, **but not symmetric**, so not a true distance).

Variational Inference Recap — ELBO

- The KL divergence can be written as

$$\mathrm{KL}(q(\mathbf{f};\boldsymbol{\theta})||p(\mathbf{f}|\mathbf{y})) = -\mathbb{E}_{q(\mathbf{f};\boldsymbol{\theta})}[\log p(\mathbf{y}|\mathbf{f})] + \mathrm{KL}(q(\mathbf{f};\boldsymbol{\theta})||p(\mathbf{f})) + \log p(\mathbf{y})$$

- Evidence $p(\mathbf{y})$ is independent of $\boldsymbol{\theta}$, so can be ignored when optimising $\boldsymbol{\theta}$.
- Negating, resulting objective is called the **Evidence Lower Bound (ELBO)**:

$$\mathrm{ELBO}(\boldsymbol{\theta}) = \mathbb{E}_{q(\mathbf{f};\boldsymbol{\theta})}[\log p(\mathbf{y}|\mathbf{f})] - \mathrm{KL}(q(\mathbf{f};\boldsymbol{\theta})||p(\mathbf{f}))$$

- Name comes from rearranging the previous equation:

$$\log p(\mathbf{y}) = \mathrm{ELBO}(\boldsymbol{\theta}) + \underbrace{\mathrm{KL}(q(\mathbf{f};\boldsymbol{\theta})||p(\mathbf{f}|\mathbf{y}))}_{\geq 0} \geq \mathrm{ELBO}(\boldsymbol{\theta})$$

Gaussian Approximate Posterior

- For our GP classification, we choose a **Gaussian approximate posterior**:

$$q(\mathbf{f}; \boldsymbol{\theta}) = N(\mathbf{f} \mid \boldsymbol{\mu}_q, \boldsymbol{\Sigma}_q).$$

- Our prior is Gaussian, so this choice will make many integrals easier.
- $\boldsymbol{\theta} = \{\boldsymbol{\mu}_q, \boldsymbol{\Sigma}_q\}$ are the **variational parameters** to be optimised via the ELBO.
- Note that this is different from a Laplace approximation, which specifically sets $\boldsymbol{\mu}_q$ to the MAP, and $\boldsymbol{\Sigma}_q$ to the inverse Hessian at the MAP.

Gaussian Approximate Posterior — ELBO

- We set $\boldsymbol{\theta}$ by maximising the ELBO, which has two terms:

$$\text{ELBO}(\boldsymbol{\theta}) = \underbrace{\mathbb{E}_{q(\mathbf{f};\boldsymbol{\theta})}[\log p(\mathbf{y}|\mathbf{f})]}_{\text{Expected Log-Likelihood (ELL)}} - \underbrace{\text{KL}(q(\mathbf{f};\boldsymbol{\theta})||p(\mathbf{f}))}_{\text{KL Divergence}}$$

- The **expected log-likelihood (ELL)** tells us how well the approximate posterior explains the data.
- The **KL divergence term** encourages the approximate posterior to stay close to the prior — acts as a regulariser.

KL Divergence between two Gaussians

$$\mathrm{KL}(q(\mathbf{f};\boldsymbol{\theta})||p(\mathbf{f})) = \int q(\mathbf{f};\boldsymbol{\theta}) \log \frac{q(\mathbf{f};\boldsymbol{\theta})}{p(\mathbf{f})} d\mathbf{f}$$

- KL divergence between two N-D multivariate Gaussians has **closed form**:

$$\mathrm{KL}(N(\mathbf{f}|\boldsymbol{\mu}_1, \boldsymbol{\Sigma}_1)||N(\mathbf{f}|\boldsymbol{\mu}_2, \boldsymbol{\Sigma}_2))$$
$$= \frac{1}{2}\left(\mathrm{tr}(\boldsymbol{\Sigma}_2^{-1}\boldsymbol{\Sigma}_1) + (\boldsymbol{\mu}_2 - \boldsymbol{\mu}_1)^T\boldsymbol{\Sigma}_2^{-1}(\boldsymbol{\mu}_2 - \boldsymbol{\mu}_1) - N + \log\frac{|\boldsymbol{\Sigma}_2|}{|\boldsymbol{\Sigma}_1|}\right)$$

- We will **assume a zero-mean prior** $p(\mathbf{f}) = N(\mathbf{f}|\mathbf{0}, \mathbf{K})$, so in our case:

$$\mathrm{KL}(N(\mathbf{f}|\boldsymbol{\mu}_q, \boldsymbol{\Sigma}_q)||N(\mathbf{f}|\mathbf{0}, \mathbf{K})) = \frac{1}{2}\left(\mathrm{tr}(\mathbf{K}^{-1}\boldsymbol{\Sigma}_q) + \boldsymbol{\mu}_q^T\mathbf{K}^{-1}\boldsymbol{\mu}_q - N + \log\frac{|\mathbf{K}|}{|\boldsymbol{\Sigma}_q|}\right)$$

- Here, $\mathbf{K}$ is shorthand for $\mathbf{K}(\mathbf{X}, \mathbf{X})$ (very common in GP literature).

Expected Log Likelihood — Monte Carlo Averaging

- The expected log-likelihood (ELL) term is an intractable integral:

$$\mathbb{E}_{q(\mathbf{f};\boldsymbol{\theta})}[\log p(\mathbf{y}|\mathbf{f})] = \int q(\mathbf{f};\boldsymbol{\theta}) \log p(\mathbf{y}|\mathbf{f}) d\mathbf{f}$$

- Approximate using **Monte Carlo averaging**.
- Take M samples from $q(\mathbf{f};\boldsymbol{\theta})$, and average the log-likelihoods:

$$\mathbb{E}_{q(\mathbf{f};\boldsymbol{\theta})}[\log p(\mathbf{y}|\mathbf{f})] \approx \frac{1}{M} \sum_{m=1}^{M} \log p(\mathbf{y}|\mathbf{f}^{(m)}) \quad \text{where } \mathbf{f}^{(m)} \sim q(\mathbf{f};\boldsymbol{\theta})$$

$$\approx \frac{1}{M} \sum_{m=1}^{M} \sum_{n=1}^{N} \left(y_n \log \sigma(f_n^{(m)}) + (1 - y_n) \log(1 - \sigma(f_n^{(m)})) \right)$$

ELL Monte Carlo Average — Reparametrisation Trick

- We will be using gradient ascent to maximise the ELBO.
- This requires us to **differentiate through a sample** from $q(\mathbf{f};\boldsymbol{\theta}) = N(\mathbf{f}\,|\,\boldsymbol{\mu}_q, \boldsymbol{\Sigma}_q)$. It isn't obvious what this means.
- **Reparametrisation trick:** rewrite the random sample as a deterministic function of parameters and standard Gaussian noise.
- Letting $\Sigma_q = \mathbf{L}\mathbf{L}^T$ (Cholesky decomposition), we write:

$$\mathbf{f}^{(s)} = \boldsymbol{\mu}_q + \mathbf{L}\mathbf{z}^{(s)} \quad \text{where } \mathbf{z}^{(s)} \sim N(\mathbf{0}, \mathbf{I})$$

- Derivative w.r.t. $\boldsymbol{\mu}_q, \boldsymbol{\Sigma}_q$ is now well-defined.

Predicting New Datapoints

- We can now compute the ELBO and use gradient ascent to find $\boldsymbol{\theta} = \{\boldsymbol{\mu}_q, \boldsymbol{\Sigma}_q\}$.
- How do we use this to do **predictions at new inputs** $\mathbf{X}_*$?
- The **true** predictive distribution looks like:

$$p(y_*|\mathbf{y}) = \int p(y_*|\mathbf{f}_*)p(\mathbf{f}_*|\mathbf{y})d\mathbf{f}_*$$

- We replace the true predictive posterior $p(\mathbf{f}_*|\mathbf{y})$ with an **approximate predictive posterior** $q(\mathbf{f}_*;\boldsymbol{\theta})$ (note the asterisk!):

$$p(y_*|\mathbf{y}) \approx \int p(y_*|\mathbf{f}_*)q(\mathbf{f}_*;\boldsymbol{\theta})d\mathbf{f}_*$$

- Need to figure out $q(\mathbf{f}_*;\boldsymbol{\theta})$ based on our variational posterior $q(\mathbf{f};\boldsymbol{\theta})$

Latent Prediction — Gaussian Integral

- $q(\mathbf{f}_*; \boldsymbol{\theta})$ approximates $p(\mathbf{f}_* | \mathbf{y})$ (the posterior latent at unseen points $\mathbf{X}_*$)
- How to obtain $q(\mathbf{f}_*; \boldsymbol{\theta})$? Marginalise prior conditional $p(\mathbf{f}_* | \mathbf{f})$ over approximate posterior $q(\mathbf{f}; \boldsymbol{\theta})$:

$$q(\mathbf{f}_*; \boldsymbol{\theta}) = \int p(\mathbf{f}_* | \mathbf{f}) q(\mathbf{f}; \boldsymbol{\theta}) d\mathbf{f}$$

- Since both $p(\mathbf{f}_* | \mathbf{f})$ and $q(\mathbf{f}; \boldsymbol{\theta})$ are Gaussian, the **result is also Gaussian**:

$$q(\mathbf{f}_*; \boldsymbol{\theta}) = N(\mathbf{f}_* | \boldsymbol{\mu}_{q_*}, \boldsymbol{\Sigma}_{q_*})$$

- Hence we just need to compute the **mean** $\boldsymbol{\mu}_{q_*}$ and **covariance** $\boldsymbol{\Sigma}_{q_*}$

Latent Prediction — Mean

- The mean $\boldsymbol{\mu}_{q_*}$ is defined as the expectation of $\mathbf{f}_*$ under $q(\mathbf{f};\boldsymbol{\theta})$.
- Using the **law of total expectation**:

$$
\begin{aligned}
\boldsymbol{\mu}_{q_*} &= \mathbb{E}_{q(\mathbf{f};\boldsymbol{\theta})}[\mathbf{f}_*] \\
&= \mathbb{E}_{q(\mathbf{f};\boldsymbol{\theta})}[\mathbb{E}[\mathbf{f}_* | \mathbf{f}]] \\
&= \mathbb{E}_{q(\mathbf{f};\boldsymbol{\theta})}[\mathbf{K}_*^T \mathbf{K}^{-1} \mathbf{f}] \\
&= \mathbf{K}_*^T \mathbf{K}^{-1} \boldsymbol{\mu}_q
\end{aligned}
$$

- where

$$
\begin{aligned}
\mathbf{K} &= \mathbf{K}(\mathbf{X}, \mathbf{X}) \\
\mathbf{K}_* &= \mathbf{K}(\mathbf{X}, \mathbf{X}_*) \\
\mathbf{K}_{**} &= \mathbf{K}(\mathbf{X}_*, \mathbf{X}_*) \qquad \text{(not used on this slide)}
\end{aligned}
$$

Latent Prediction — Covariance

- Similarly, using the **law of total variance**:

$$
\begin{aligned}
\boldsymbol{\Sigma}_{q_*} &= \mathrm{Var}_{q(\mathbf{f};\boldsymbol{\theta})}[\mathbf{f}_*] \\
&= \mathbb{E}_{q(\mathbf{f};\boldsymbol{\theta})}[\mathrm{Var}[\mathbf{f}_* | \mathbf{f}]] + \mathrm{Var}_{q(\mathbf{f};\boldsymbol{\theta})}[\mathbb{E}[\mathbf{f}_* | \mathbf{f}]] \\
&= \mathbb{E}_{q(\mathbf{f};\boldsymbol{\theta})}[\mathbf{K}_{**} - \mathbf{K}_*^T \mathbf{K}^{-1} \mathbf{K}_*] + \mathrm{Var}_{q(\mathbf{f};\boldsymbol{\theta})}[\mathbf{K}_*^T \mathbf{K}^{-1} \mathbf{f}] \\
&= \mathbf{K}_{**} - \mathbf{K}_*^T \mathbf{K}^{-1} \mathbf{K}_* + \mathbf{K}_*^T \mathbf{K}^{-1} \mathrm{Var}_{q(\mathbf{f};\boldsymbol{\theta})}[\mathbf{f}] \mathbf{K}^{-1} \mathbf{K}_* \\
&= \mathbf{K}_{**} - \mathbf{K}_*^T \mathbf{K}^{-1} \mathbf{K}_* + \mathbf{K}_*^T \mathbf{K}^{-1} \boldsymbol{\Sigma}_q \mathbf{K}^{-1} \mathbf{K}_*
\end{aligned}
$$

- where

$$
\begin{aligned}
\mathbf{K} &= \mathbf{K}(\mathbf{X}, \mathbf{X}) \\
\mathbf{K}_* &= \mathbf{K}(\mathbf{X}, \mathbf{X}_*) \\
\mathbf{K}_{**} &= \mathbf{K}(\mathbf{X}_*, \mathbf{X}_*)
\end{aligned}
$$

Label Prediction — Monte Carlo Averaging

- We now have $q(\mathbf{f}_*; \boldsymbol{\theta})$ by the results on slides 16, 17, and 18. But the prediction is still intractable (non-conjugate likelihood $p(y_*|\mathbf{f}_*)$):

$$p(y_*|\mathbf{y}) \approx \int p(y_*|\mathbf{f}_*)q(\mathbf{f}_*; \boldsymbol{\theta})d\mathbf{f}_*$$

- Similar to the ELL, approximate using **Monte Carlo averaging**:

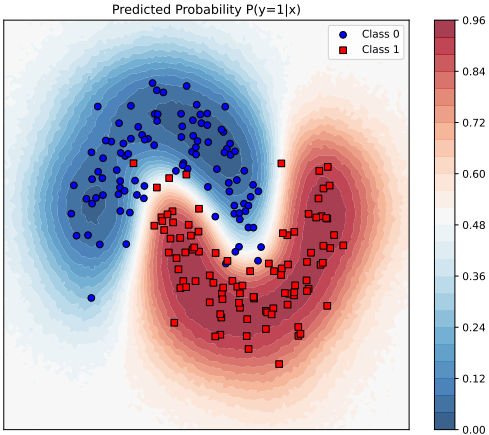
$$p(y_*|\mathbf{y}) \approx \frac{1}{S}\sum_{s=1}^{S} p(y_*|\mathbf{f}_*^{(s)}) \quad \text{where } \mathbf{f}_*^{(s)} \sim q(\mathbf{f}_*; \boldsymbol{\theta})$$

- Concretely with sigmoid:

$$\approx \frac{1}{S}\sum_{s=1}^{S} \sigma(f_*^{(s)})$$

And breathe...

Toy Example — Predictions

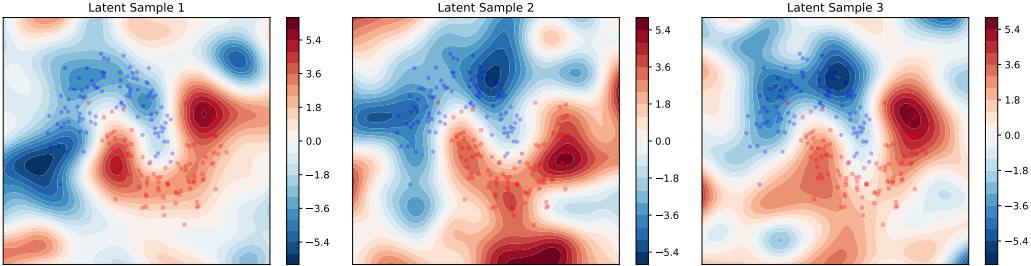


- Train $q(\mathbf{f}; \boldsymbol{\theta})$ by maximising ELBO (I used the Adam optimiser).
- Use slides 16, 17, and 18 to get $q(\mathbf{f}_*; \boldsymbol{\theta}; \mathbf{X}_*)$ where $\mathbf{X}_*$ is a grid.
- Plot predicted class probability:

$$p(y_* = 1 \,|\, \mathbf{y}) \approx \frac{1}{S} \sum_{s=1}^{S} \sigma(f_*^{(s)})$$

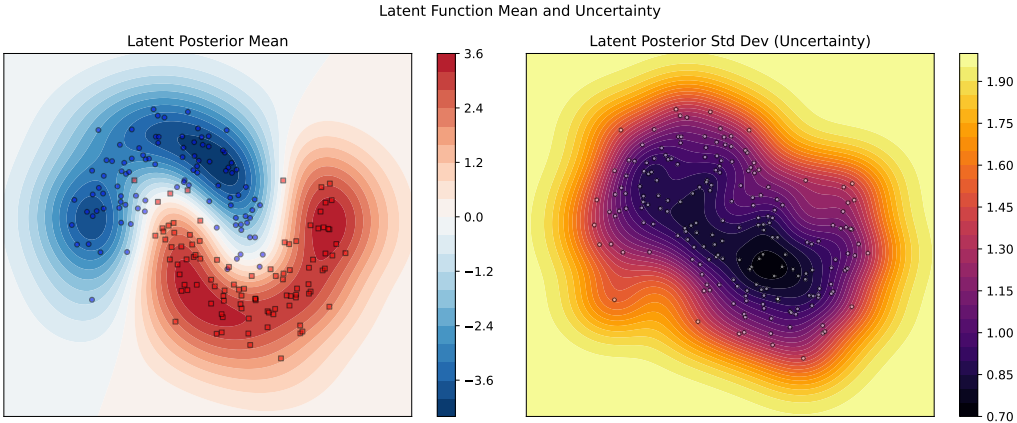
Toy Example — Latent Function Samples

- Here, we plot 3 samples of the **latents** from $q(\mathbf{f}_*; \boldsymbol{\theta})$
- They vary quite a bit, but generally assign small values to class 0 regions, and large values to class 1 regions.



Toy Example — Mean and Uncertainty

- Here we plot $\boldsymbol{\mu}_{q_*}$ and the standard deviation $\mathrm{diag}(\boldsymbol{\Sigma}_{q_*})^{1/2}$



Practical Tips

Numerical Stability Tricks — Jitter

- Kernel matrices can have **very small eigenvalues**, causing linear algebra routines to **fail** (NaNs, errors).
- **Double precision** floating points help.
- It is common to add a small constant value to the diagonal:

$$\mathbf{K} \leftarrow \mathbf{K} + \epsilon \mathbf{I}$$

- Typical values: $\epsilon = 10^{-6}$ or 10^{-8}
- This is called **jitter** or **Tikhonov regularisation**.

Numerical Stability Tricks — Matrix Inverses

- Matrix inverses times vectors are equivalent to **solving** linear systems:

$$\mathbf{x} = \mathbf{K}^{-1}\mathbf{v} \quad \Leftrightarrow \quad \mathbf{K}\mathbf{x} = \mathbf{v}$$

- Don't compute the inverse explicitly:
- `invK = np.linalg.inv(K); result = invK @ v`
- **Solve the linear system instead** (it's more stable):
- `result = np.linalg.solve(K, v)`
- For positive-definite matrices (like kernels), use the **Cholesky decomposition** for better stability and speed:
- `L = np.linalg.cholesky(K)`
- `invK = scipy.linalg.cho_solve((L, True), v)`

Numerical Stability Tricks — Log Determinants

- **Log-determinants** appear in the KL divergence term.
- For positive-definite matrices, use the Cholesky decomposition:

$$\log |\mathbf{K}| = 2 \sum_{i=1}^{N} \log L_{ii} \quad \text{where } \mathbf{K} = \mathbf{L}\mathbf{L}^T \text{ (Cholesky)}$$

Whitened Parametrisation

- The algorithm we presented often **struggles to optimise properly** (e.g. **underfitting**, latents don't match data).
- Common fix — **whitened parametrisation**:

$$\mathbf{f} = \mathbf{L}\mathbf{z} \quad \text{with prior} \quad \mathbf{z} \sim N(\mathbf{0}, \mathbf{I}), \quad \mathbf{L} = \text{chol}(\mathbf{K})$$

- Variational distribution over $\mathbf{z}$ ($N(\mathbf{0}, \mathbf{I})$ prior = more uniform landscape):

$$q(\mathbf{z}; \boldsymbol{\theta}) = N(\mathbf{z} | \boldsymbol{\mu}_z, \boldsymbol{\Sigma}_z)$$

- ELBO becomes:

$$\text{ELBO}(\boldsymbol{\theta}) = \mathbb{E}_{q(\mathbf{z};\boldsymbol{\theta})}[\log p(\mathbf{y} | \mathbf{f} = \mathbf{L}\mathbf{z})] - \text{KL}(q(\mathbf{z}; \boldsymbol{\theta}) || N(\mathbf{z} | \mathbf{0}, \mathbf{I}))$$

- Replace mean and covariance in previous calculations:

$$\boldsymbol{\mu}_f = \mathbf{L}\boldsymbol{\mu}_z, \quad \boldsymbol{\Sigma}_f = \mathbf{L}\boldsymbol{\Sigma}_z\mathbf{L}^T$$

Hyperparameter Tuning

- The ELBO is a **lower bound to log-evidence** $\log p(\mathbf{y})$
- Maximising ELBO w.r.t. kernel hyperparameters therefore approximates **type-II maximum likelihood**.
- Other strategies if this isn't working very well:
  - Cross-validation
  - Grid search
  - Random search
  - Fixed values based on prior knowledge
- Note that, for separable classes, the **output variance** hyperparameter may grow arbitrarily large.

Other Approximations

- If you just want GP classification, other approximations are available:
  - Expectation Propagation (EP)
  - Laplace Approximation
  - Markov Chain Monte Carlo (MCMC)
  - And many more...
- But the VI approach underpins the **sparse GPs** we will see next lecture.

Conclusion

Takeaways

- Classification with GPs requires **non-conjugate likelihoods**.
- No closed-form posterior.
- **Variational inference** is one approach to approximating the posterior.
- **Gaussian approximate posterior** leads to many simplifications.
- **Monte Carlo averaging** and the **reparametrisation trick** help with intractable integrals.

Next Lecture

- The GPs covered this week require us to compute and invert $N \times N$ kernel matrices.
- This requires $O(N^3)$ time and $O(N^2)$ memory — **impractical for large datasets**.
- Next lecture, we will cover **sparse Gaussian processes**, which address this problem.

References and Further Reading

References and Further Reading

- Carl Edward Rasmussen, Christopher K. I. Williams. *Gaussian Processes for Machine Learning*. MIT Press, 2006. `http://www.gaussianprocess.org/gpml/`
- Leibfried et. al. 2022 *A Tutorial on Sparse Gaussian Processes and Variational Inference*. `https://arxiv.org/abs/2012.13962`
- Alexander Graeme de Garis Matthews *Scalable Gaussian process inference using variational methods*. University of Cambridge, 2016. `https://www.repository.cam.ac.uk/items/a2c7e9a7-d7bb-40c0-9285-8e0f487869e1`